

Rapport de projet

Classification de paroles de chansons

Rap vs Variété Française - NLP & scikit-learn

*Pipeline de text mining : EDA, prétraitement, TF-IDF, modélisation supervisée,
comparaison zero-shot et démonstrateur interactif*

Sommaire

Sommaire	2
1. Contexte et objectif.....	3
2. Données.....	3
3. Méthodologie	3
3.1 Prétraitement	3
3.2 Vectorisation	3
3.3 Modèles comparés.....	3
3.4 Validation croisée groupée par artiste	3
4. Anomalie détectée et corrigée.....	4
5. Résultats	4
6. Comparaison avec un modèle pré-entraîné (zero-shot)	5
7. Architecture du projet et outillage	5
7.1 Structure du code.....	5
7.2 Démonstrateur interactif (Streamlit)	5
8. Limites et pistes d'amélioration	6
9. Conclusion	6

1. Contexte et objectif

Ce projet construit un pipeline de text mining permettant de classer automatiquement des paroles de chansons en deux catégories : Rap et Variété Française (VF). Il couvre l'ensemble de la chaîne : analyse exploratoire des données, prétraitement NLP, vectorisation TF-IDF, modélisation supervisée, évaluation critique, comparaison à un modèle pré-entraîné, et mise à disposition via une démonstration interactive.

Au-delà du résultat brut, l'objectif de ce rapport est de documenter la démarche de fiabilisation du projet : une anomalie méthodologique majeure a été identifiée et corrigée au cours du travail, et sont présentées en détail en section 4.

2. Données

Les données sont fournies sous forme de fichiers CSV par artiste (un fichier par artiste, colonne lyrics contenant les paroles), consolidés en un dataset unique.

Nombre total de chansons	2 633
Variété Française (VF)	1 718 ($\approx 65\%$)
Rap	915 ($\approx 35\%$)
Artistes Rap	13 (Akhenaton, Booba, IAM, JUL, Nekfeu, Orelsan...)
Artistes VF	13 (Piaf, Brel, Aznavour, Cabrel, Dassin...)

Anomalie de donnée détectée :

Le fichier data/chansons/VF/Georges Brassens.csv existe mais contient 0 ligne : échec de collecte à la source (scraping), pas un bug du pipeline de chargement. Cet artiste est donc absent du dataset consolidé et des 13 artistes VF listés ci-dessus.

3. Méthodologie

3.1 Prétraitement

Normalisation du texte (minuscule, normalisation des espaces), retrait des mots outils (stopwords) combinant la liste NLTK française et une liste personnalisée du projet (data/stopword.txt).

3.2 Vectorisation

Représentation TF-IDF (Term Frequency - Inverse Document Frequency) sur unigrammes, avec un seuil de fréquence minimale (min_df) optimisé par validation croisée.

3.3 Modèles comparés

Quatre familles de modèles ont été comparées via optimisation bayésienne des hyperparamètres (BayesSearchCV, scikit-optimize) : LinearSVC, régression logistique, RandomForest et XGBoost. Le RandomForest s'est montré le plus performant.

3.4 Validation croisée groupée par artiste

Le split entraînement/test et la validation croisée interne sont groupés par artiste (StratifiedGroupKFold) : aucun artiste n'apparaît à la fois dans le train et le test. Cette décision méthodologique, et la raison pour laquelle elle a été introduite, sont détaillées en section 4.

4. Anomalie détectée et corrigée

Le travail réalisé sur ce projet a délibérément inclus une phase d'audit critique.

Fuite de données au niveau artiste (data leakage) :

Le split train/test initial était effectué chanson par chanson. Un même artiste pouvait donc apparaître à la fois dans le jeu d'entraînement et le jeu de test, permettant au modèle d'apprendre une « signature d'artiste » (vocabulaire, thèmes récurrents) plutôt qu'un style de genre musical, gonflant artificiellement le score.

Correction :

Passage à un split et une validation croisée groupés par artiste (StratifiedGroupKFold). Conséquence observée : l'écart-type de la validation croisée est passé de $\pm 0,0076$ (trompeusement stable) à $\pm 0,0551$. Une variance bien plus réaliste de l'incertitude de généralisation à de nouveaux artistes.

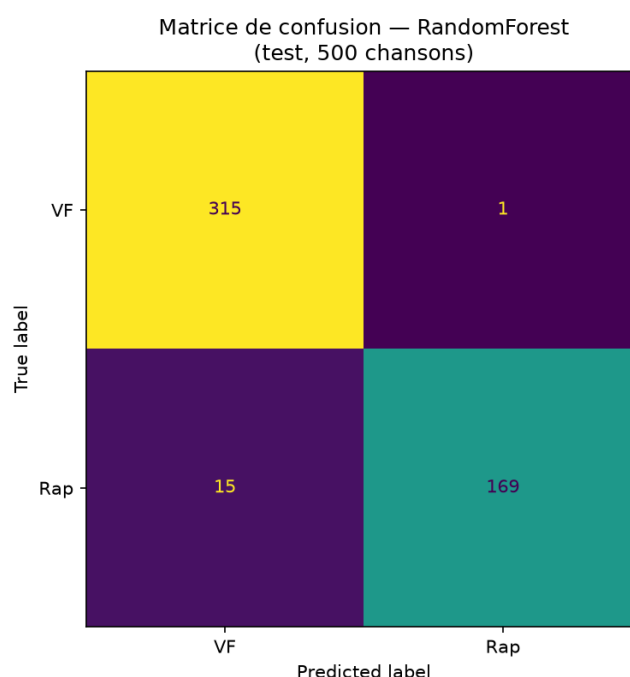
5. Résultats

Meilleur modèle retenu (sélection par validation croisée, métrique F1 macro) : RandomForest sur représentation TF-IDF.

Version	F1 macro (test)	F1 macro CV ($\pm$ std)
Split par chanson (fuite d'artiste)	0,9632	$0,9486 \pm 0,0076$
Split par artiste (bug stopwords)	0,9606	$0,9203 \pm 0,0551$
Split par artiste + stopwords corrigés	0,965	$0,9198 \pm 0,0423$

Performances finales sur le jeu de test (6 artistes jamais vus à l'entraînement : Diam's, Johnny Hallyday, Kaaris, Nekfeu, Sinik, Vald) : F1 macro 0,965 - F1 (Rap) 0,9548 et Accuracy 0,97.

Les classes étant déséquilibrées (VF > Rap), le F1 macro est privilégié comme métrique principale, en complément de l'accuracy.



Matrice de confusion - RandomForest, jeu de test (500 chansons)

en barres des termes les plus influents dans la décision (deux visualisations complémentaires présentées sous forme d'onglets pour ne pas alourdir la page).

L'apparence visuelle est définie par un thème natif Streamlit (.streamlit/config.) plutôt que par du CSS injecté, conformément aux bonnes pratiques Streamlit.

Démo en ligne : [*Rap vs Variété française · Streamlit*](#)

8. Limites et pistes d'amélioration

- Biais de longueur : les paroles de Rap restent en moyenne plus longues que celles de la Variété Française, ce qui peut faciliter artificiellement la séparation des classes.
- Diversité d'artistes limitée au test : avec seulement 13 artistes par genre, le split de test retenu ne contient qu'un seul artiste VF (Johnny Hallyday). Le score de validation croisée (moyenné sur 5 découpages) reste l'estimation la plus fiable de la généralisation.
- Limite du zero-shot : les paroles sont tronquées en interne à ~512 tokens par le tokenizer, ce qui limite l'information disponible pour les chansons longues.
- Pistes futures : calibration des scores de probabilité, comparaison avec des modèles linéaires régularisés, représentation par n-grammes de caractères.

9. Conclusion

Le modèle supervisé TF-IDF + RandomForest atteint un F1 macro de 0,965 sur un jeu de test composé d'artistes jamais vus à l'entraînement, confirmant l'existence d'un signal linguistique fortement discriminant entre Rap et Variété Française. Au-delà de ce résultat, le travail de fiabilisation mené, détection et correction d'une fuite de données, constitue une démonstration de rigueur méthodologique aussi importante que la performance du modèle elle-même.